

Ch 7. Les procédures et les fonctions

Synthèse du Chapitre 7 : Les procédures et les fonctions en PL/SQL

Ce chapitre traite des **procédures** et des **fonctions** en PL/SQL, qui sont des sous-programmes permettant de modulariser le code, de le réutiliser et de l'optimiser. Les procédures et les fonctions sont des blocs nommés qui peuvent être appelés à plusieurs endroits dans un programme. Elles diffèrent principalement par le fait que les fonctions retournent une valeur, tandis que les procédures ne retournent rien (ou plutôt, elles peuvent retourner des valeurs via des paramètres **OUT** ou **IN OUT**).

1. CREATE PROCEDURE

Une **procédure** est un bloc nommé qui peut contenir des instructions SQL et PL/SQL. Elle peut accepter des paramètres en entrée (**IN**), en sortie (**OUT**), ou en entrée/sortie (**IN OUT**).

- **Syntaxe de création d'une procédure :**

```
CREATE OR REPLACE PROCEDURE NomDeLaProcedure (param1 IN type1, param2 OUT type2) IS
BEGIN
    -- Instructions
EXCEPTION
    -- Gestion des exceptions
END;
```

- **Exemple de procédure :**

```
CREATE OR REPLACE PROCEDURE Afficher (NumSecu IN Employees.NumSecu%TYPE) IS
    UnEmployee Employees%ROWTYPE;
BEGIN
    SELECT * INTO UnEmployee FROM Employees WHERE NumSecu = Afficher.NumSecu;
```

```

        DBMS_OUTPUT.PUT_LINE('Nom : ' || UnEmployee.Nom);
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            RAISE_APPLICATION_ERROR(-20001, 'Aucun employé trouvé');
    END;

```

- **Compilation et exécution :**

- La procédure est compilée et stockée dans le dictionnaire de données.
- Elle peut être exécutée avec `EXECUTE` ou dans un bloc PL/SQL.

2. CREATE FUNCTION

Une **fonction** est similaire à une procédure, mais elle retourne une valeur. Elle est souvent utilisée pour effectuer des calculs ou des transformations de données.

- **Syntaxe de création d'une fonction :**

```

CREATE OR REPLACE FUNCTION NomDeLaFonction (param1 IN type1)
RETURN return_type IS
BEGIN
    -- Instructions
    RETURN valeur;
EXCEPTION
    -- Gestion des exceptions
END;

```

- **Exemple de fonction :**

```

CREATE OR REPLACE FUNCTION Rechercher (NumSecu IN Employees.
NumSecu%TYPE) RETURN Employees%ROWTYPE IS
    UnEmployee Employees%ROWTYPE;
BEGIN
    SELECT * INTO UnEmployee FROM Employees WHERE NumSecu = R
echercher.NumSecu;
    RETURN UnEmployee;
EXCEPTION
    WHEN NO_DATA_FOUND THEN

```

```
RAISE_APPLICATION_ERROR(-20002, 'Aucun employé trouvé');  
END;
```

- **Exécution d'une fonction :**

```
DECLARE  
    UnEmployee Employees%ROWTYPE;  
BEGIN  
    UnEmployee := Rechercher('121212');  
    DBMS_OUTPUT.PUT_LINE('Nom : ' || UnEmployee.Nom);  
EXCEPTION  
    WHEN OTHERS THEN  
        DBMS_OUTPUT.PUT_LINE('Erreur : ' || SQLERRM);  
END;
```

3. Utiliser les paramètres avec NOCOPY

Le mot-clé `NOCOPY` permet de passer les paramètres `OUT` et `IN OUT` par référence plutôt que par valeur, ce qui améliore les performances, surtout pour les collections ou les structures de données volumineuses.

- **Exemple :**

```
CREATE OR REPLACE PROCEDURE NeFaitRien_v2 (p_EmployeTab IN O  
UT NOCOPY EmployeTabType) IS  
BEGIN  
    NULL;  
END;
```

- **Attention :** L'utilisation de `NOCOPY` peut entraîner des effets indésirables en cas d'exception, car les modifications des paramètres peuvent ne pas être annulées.

4. RAISE_APPLICATION_ERROR

La procédure `RAISE_APPLICATION_ERROR` permet de lever des erreurs personnalisées avec un code d'erreur et un message.

- **Syntaxe :**

```
RAISE_APPLICATION_ERROR(code_erreur, message);
```

- **Exemple :**

```
RAISE_APPLICATION_ERROR(-20001, 'Aucun employé trouvé');
```

5. Les notations nommées

Les notations nommées permettent de spécifier les paramètres dans un ordre différent de celui de la déclaration, ce qui est utile lorsque certains paramètres ont des valeurs par défaut.

- **Exemple :**

```
Search_Client(p_NomClient ⇒ 'Dupont', p_AdresseClient ⇒ 'Namur');
```

6. Les paramètres par défaut

Les paramètres peuvent avoir des valeurs par défaut, ce qui permet de les omettre lors de l'appel de la procédure ou de la fonction.

- **Exemple :**

```
CREATE OR REPLACE PROCEDURE Select_Client (Nom IN CHAR, Code  
Postal IN NUMBER := 4500) IS  
BEGIN  
    -- Instructions  
END;
```

- **Appel :**

```
Select_Client('Dupont'); -- Utilise la valeur par défaut pour CodePostal
```

7. Les procédures autonomes

Les procédures autonomes sont des procédures qui s'exécutent dans une transaction indépendante. Elles sont utiles pour enregistrer des informations de journalisation ou de trace sans affecter la transaction principale.

- **Syntaxe :**

```
CREATE OR REPLACE PROCEDURE Debug (Message IN VARCHAR2) IS
  PRAGMA AUTONOMOUS_TRANSACTION;
BEGIN
  INSERT INTO Trace (Ligne) VALUES (Message);
  COMMIT;
END;
```

- **Exemple d'utilisation :**

```
BEGIN
  INSERT INTO Clients (refclient, nomclient, adresseclient, localite)
  VALUES (1, 'Gramme', 'Rue de la dynamo', 'Rome');
  COMMIT;
EXCEPTION
  WHEN DUP_VAL_ON_INDEX THEN
    Debug('Tentative d\\'insertion d\\'un client existant');
    ROLLBACK;
END;
```

Conclusion

Les **procédures** et les **fonctions** sont des éléments essentiels en PL/SQL pour structurer et modulariser le code. Elles permettent de regrouper des instructions logiques, de gérer les erreurs, et d'optimiser les performances grâce à des techniques comme **NOCOPY** et les transactions autonomes. Une bonne compréhension de ces concepts est cruciale pour développer des applications PL/SQL robustes et efficaces.